Thetawave.ai

5.3 景點 - 指令模式 _ 解耦操作及能力.mp4

🕹️ 指令模式(Command Pattern)介紹與應用場景

> **指令模式**的核心是將「每一道指令封裝成一個物件」，使得指令的發出者（Invoker）與接收者（Receiver）能夠解耦，並提升程式的彈性與可維護性。

實際生活比喻

指令模式類似將遙控器上的每個按鈕視為「指令插槽」，而每個指令是一個獨立的「指令物件」。按下按鈕時，遙控器只負責呼叫對應的指令物件執行，不需要知道指令的具體執行細節。

🏠 遙控器案例說明

初版需求與問題點

按鈕編號	操作說明	控制家具
0	電風扇調整到下一個風力等級	電風扇(Fan)
1	電風扇調整到上一個風力等級	電風扇(Fan)
2	冷氣開啟	冷氣(Air Conditioner)
3	冷氣關閉	冷氣(Air Conditioner)
4	電視開啟	電視(Television)
5	電視關閉	電視(Television)

- 遙控器(Controller)中使用 switch case 直接寫死按鈕與操作的對應關係。
- **問題**：當需求改變或新增家具時，程式碼會爆炸（修改難度大，不易維護）。

🔄 第二版需求：彈性綁定指令

- 希望**能夠在外部輕易更改按鈕與指令的綁定規則**，符合**開閉原則(Open-Close Principle, OCP)**。
- 不允許修改遙控器內部的程式碼，只能透過新增或替換指令物件來實現新功能。

🧩 指令模式的解決方案

核心思想

> **將每一道具體指令封裝成一個物件（Concrete Command），遙控器只負責呼叫這個指令物件的執行方法（Execute），不直接操作家具（Receiver）。**

對應角色

Thetawave.ai

角色名稱	說明	遙控器案例中實體
Client	建立並設定指令物件	Main Method 中實例化指令與家具
Invoker	發出指令（呼叫指令物件）	Controller 類別（遙控器）
Command	指令介面，定義執行方法	Command Interface，含 Execute 方法
Concrete Command	具體指令類別，實作 Command 介面	Turn On AC Command, Fan Next Level Command 等
Receiver	真正執行動作的對象	Furniture 物件，如 Air Conditioner, Fan, Television

---

🛠️ 指令模式實作三步驟

步驟	說明
1. 封裝變動處	將 6 道不同家具指令從 Controller 中抽離，封裝成獨立類別
2. 抽象共同行為	萃取出 Command 介面，定義統一的 Execute() 方法供所有指令實作
3. 委派責任	Controller 不再直接控制家具，改為持有 Command 陣列，按鈕執行對應 Command 的 Execute()

📌 指令介面與具體指令範例

```
// 指令介面
interface Command {
  void Execute();
}

// 具體指令範例：電風扇調整到下一個風力等級
class FanNextLevelCommand implements Command {
  private Fan fan;
  public FanNextLevelCommand(Fan fan) { this.fan = fan; }
  public void Execute() { fan.nextLevel(); }
}
```

- 每個具體指令都**依賴注入(Dependency Injection)**其控制的家具物件。
- Execute()方法封裝了具體的操作細節。

🔘 Controller 類別改寫

功能	描述
指令陣列	持有 Command 陣列，對應 6 個按鈕的指令插槽
設定指令方法	提供 SetCommand(int buttonIndex, Command command) 讓 Client 設定按鈕指令
按鈕執行方法	PressButton(int buttonIndex) 呼叫該按鈕對應的 Command 的 Execute()

- 不再需要知道家具細節，與具體操作完全解耦。
- 方便替換、擴充指令物件。

Thetawave.ai

💡 指令模式的設計優點

優點	說明
解耦 Invoker 與 Receiver	遙控器不認識家具物件，只知道 Command 介面
符合開閉原則 (OCP)	新增或修改指令不需更改遙控器程式碼
易於維護	各指令封裝獨立，錯誤可快速定位於具體指令類別
支援還原(Undo)與重做(Redo)	在 Command 介面新增 Undo() 方法，實作指令的反向操作，搭配堆疊結構實現 Undo/Redo 功能

↩️ ↪️ Undo 與 Redo 功能設計

Undo/Redo 演算法核心

規則編號	規則說明
1	執行新指令後，將該指令推入堆疊 S1，並清空堆疊 S2
2	Undo 時，從 S1 彈出指令，呼叫該指令的 Undo()，再推入堆疊 S2
3	Redo 時，從 S2 彈出指令，呼叫該指令的 Execute()，再推入堆疊 S1

Undo 舉例

- **Turn On Air Conditioner Command** 的 Execute() 是「開啟冷氣」
- 對應的 Undo() 是「關閉冷氣」

🏷️ 指令模式關鍵詞總整理

名稱	中文解釋	角色功能與說明
Command	指令介面	定義執行指令的介面，通常含有 Execute() 方法
Concrete Command	具體指令類別	實作 Command，封裝具體的動作與 Receiver
Invoker	調用者（如遙控器）	呼叫 Command 的 Execute()，不關心具體動作
Receiver	接收者（被控制家具）	實際執行指令動作的物件，如冷氣、電風扇、電視
Client	用戶端	建立指令、接收者與調用者，並設定指令與按鈕的綁定
依賴注入(DI)	依賴注入	將 Receiver 物件注入 Concrete Command，促進鬆耦合
開閉原則(OCP)	對修改封閉、對擴充開放	新增指令不需修改既有程式碼，提高系統彈性

🎯 指令模式的應用時機與意義

- **當應用程式中有「操作」與「指令」需要彈性綁定時，指令模式是一個理想解決方案。**
- 透過將指令封裝成物件，能有效降低系統耦合度，提升擴充性與維護性。
- 系統可以輕易實現複雜功能，如還原（Undo）、重做（Redo）、排程指令等。

指令模式透過將指令封裝為物件，讓調用者與執行者徹底解耦，促使系統具備良好彈性與維護性。這種結構使得系統能隨時新增或替換指令，且能自然實現還原與重做功能，符合軟體設計中追求的模組化與可擴充性。

Thetawave.ai

5.5 景點 - 狀態模式_梳理複雜行為變化.mp4

🎟️ 購票機系統（Ticket System）設計需求與分析

系統功能與行為規則

- **購票機用途**：讓學員以硬幣購買門票，每張票價為 3 枚硬幣。
- **硬幣投入限制**：一次只能投入 1 枚硬幣。
- **票機狀態分為兩種**：
  - **有票存（In stock）**：可接受硬幣，根據硬幣投入數量決定是否可購票。
  - **已售完（Sold out）**：無票存，投入硬幣時會直接退回硬幣。
- **硬幣投入後判斷**：
  - 若硬幣總數 ≥ 3（價格），購票燈亮起，表示可按購買鈕。
  - 燈亮時按購買鈕可購票並重置硬幣數。
  - 燈未亮時只能繼續投入硬幣。
- **退錢功能**：按退錢鈕時，退回所有已投入硬幣，並重置硬幣數與熄燈。
- **補充票存**：維護者可補充門票至票機。

物件導向分析（OOA）

類別與屬性

屬性名稱	說明	資料型態	備註
Tickets	庫存門票數量	整數	代表票機內剩餘票數
Total	硬幣總數	整數	票機內目前投入硬幣的總數
Light on	購買燈狀態	布林值（Boolean）	True 為亮燈，False 為熄燈
Price	門票價格	整數	靜態屬性，固定為 3

操作（方法）

方法名稱	說明	公開性
Insert coin	投入一枚硬幣，更新硬幣總數或退幣	公開
Press buy button	按購買鈕，若燈亮則購買門票	公開
Press refund button	按退幣鈕，退回所有硬幣並更新狀態	公開
Fill tickets	補充門票存量	公開
Update coins display	更新螢幕顯示的硬幣總數	私有
Turn light	控制購買燈開啟或熄滅	私有
Spit coins	退幣操作	私有
Issue one ticket	發出一張門票	私有

🛠️ 重要操作行為解析

Insert coin 行為流程

1. 投入一枚硬幣，硬幣總數加 1。
2. 判斷票存是否存在：
   - **有票存且硬幣總數 ≥ 價格**：

Thetawave.ai

    - 更新螢幕顯示。
    - 亮起購買燈，不再接受硬幣。
- **有票存但硬幣不足**：
    - 更新螢幕顯示，繼續接受硬幣。
- **無票存（售完）**：
    - 直接吐出剛投入的硬幣。

Press buy button 行為流程

1. 判斷購買燈是否亮起：
    - **亮起**：
        - 扣除價格硬幣數，硬幣總數歸零。
        - 更新螢幕顯示。
        - 發出一張門票。
        - 熄滅購買燈。
    - **未亮起**：
        - 不允許購買，印出錯誤訊息。

Press refund button 行為流程

1. 退回所有硬幣。
2. 更新螢幕顯示硬幣數為零。
3. 若購買燈亮起，則熄滅燈光。

🔁 狀態機（STATE MACHINE）觀點

狀態定義與轉移

購票機存在三個主要狀態：

狀態名稱	狀態說明
Sold out	無票存，投入硬幣皆退回
In stock	有票存，硬幣尚未足夠購買
Enough coins	有票存且硬幣已足夠購買，購買燈亮起

狀態轉移規則概述

目前狀態	觸發事件	條件(Guard)	行動(Action)	下一狀態
In stock	Insert coin	Total < Price	更新硬幣顯示(update coins display)	In stock
In stock	Insert coin	Total ≥ Price	更新硬幣顯示，亮燈(turn light on)	Enough coins
Enough coins	Press buy button	Tickets > 0	發票(issue one ticket)，更新硬幣顯示，熄燈(turn light off)	In stock
Enough coins	Press buy button	Tickets = 0	發票(issue one ticket)，更新硬幣顯示，熄燈	Sold out
Sold out	Insert coin	無條件	退回硬幣(spit coins)	Sold out
Sold out	Fill tickets	補充票存	進入 In stock 狀態	In stock

Thetawave.ai

狀態機圖的元素解析

- **狀態（State）**：以圓角矩形表示，內含狀態名稱與可選的 entry action、exit action。
- **狀態轉移（Transition）**：箭頭連接兩狀態，附帶三要素：
  - **觸發事件 (Trigger)**：事件名稱，如 Insert coin。
  - **成立條件 (Guard)**：以中括號表示，如 [Total ≥ Price]。
  - **行動 (Action)**：斜線後的動作，如 update coins display。

Entry 與 Exit Action 範例

- 進入 **Enough coins** 狀態時，執行亮燈（turn light on）。
- 離開 **Enough coins** 狀態時，執行熄燈（turn light off）。

🏷️ 類別圖擴充與屬性維護

新增狀態屬性與改寫行為判斷基準

- 新增屬性：**state**，用以代表 Ticket System 的當前狀態（In stock、Sold out、Enough coins）。
- 所有方法行為改為根據 **state** 來判斷動作，而非單純屬性判斷。

屬性與狀態一致性規則

狀態	屬性條件
Sold out	Tickets = 0，Total = 0
In stock	Tickets > 0，Total < Price
Enough coins	Tickets > 0，Total = Price

Enter state 方法行為流程

步驟	說明
1. 離開當前狀態	若為 Enough coins，熄滅購買燈
2. 更新狀態	將 state 設為新狀態
3. 進入新狀態	根據狀態調整屬性與執行 entry action：
	- **Enough coins**：Total 設為 Price，購買燈亮起
	- **In stock**：若 Total ≥ Price，退回多餘硬幣
	- **Sold out**：Total 退回所有硬幣，Tickets 設為 0

💻 程式碼實作重點說明

Insert coin 方法改寫

```
def insert_coin(self):
    if self.state == "Enough coins":
        print("Error: Cannot insert more coins, already enough.")
        return
    self.total += 1
    if self.state == "In stock":
        self.update_coins_display()
        if self.total >= self.price:
            self.enter_state("Enough coins")
    elif self.state == "Sold out":
```

Thetawave.ai

```
        self.spit_coins(1)
```

- 以狀態作為行為判斷基準。
- 投入硬幣後依狀態更新或退幣。

Press buy button 方法改寫

```
def press_buy_button(self):
    if self.state != "Enough coins":
        print("Error: Not enough coins to buy ticket.")
        return
    if self.tickets > 0:
        self.total -= self.price
        self.update_coins_display()
        self.issue_one_ticket()
        if self.tickets == 0:
            self.enter_state("Sold out")
        else:
            self.enter_state("In stock")
    else:
        print("Error: No tickets left.")
```

- 僅在 Enough coins 狀態允許購買。
- 購買後依票數決定狀態轉移。

Press refund button 方法改寫

```
def press_refund_button(self):
    self.spit_coins(self.total)
    self.total = 0
    self.update_coins_display()
    if self.light_on:
        self.turn_light(False)
    if self.tickets == 0:
        self.enter_state("Sold out")
    else:
        self.enter_state("In stock")
```

- 退回所有硬幣並更新螢幕與燈光。
- 根據票存決定狀態。

🔍 測試案例解析

測試案例	操作流程描述	預期結果
Case 1	投入 3 枚硬幣後嘗試投入第 4 枚硬幣	第 4 枚硬幣被拒絕，顯示錯誤訊息
Case 2	投入 3 枚硬幣後按退錢鈕	退回 3 枚硬幣，購買燈熄滅
Case 3	投入 3 枚硬幣後按購買鈕	出票，硬幣數歸零，購買燈熄滅
Case 4	無硬幣狀態按購買鈕	顯示錯誤訊息，不允許購買
Case 5	無硬幣狀態按退錢鈕	退回 0 枚硬幣，狀態正常
Case 6	將最後一張票買完，票機售完狀態下投入硬幣	硬幣被立刻退回，燈不亮

Thetawave.ai

🔧 狀態機圖（State Machine Diagram）的實務應用

- 狀態機圖可清楚呈現複雜行為的狀態轉移和條件。
- 利用狀態機，開發者與測試工程師能更有效率地理解及測試系統行為。
- 直接切換狀態的設計，減少測試前置程序，提升測試效率。

💡 狀態模式（State Pattern）核心觀念

- 利用物件的**狀態（State）**來決定其行為，而非用大量條件判斷。
- 狀態間的轉移清晰，行為分散到各狀態，降低系統複雜度。
- 強調狀態與屬性的一致性維護，確保系統狀態準確反映內部資料。

狀態模式透過將不同狀態的行為封裝，達成行為分離與狀態管理的清晰結構，有助於系統維護與擴展。狀態轉移的明確規範使得行為流程更具可預測性，減少錯誤發生的可能，提升整體系統穩定性。

🎟️ Ticket System 的狀態模式設計

> **狀態模式（State Pattern）**：讓物件的行為隨著其內部狀態改變而改變，且讓狀態的行為封裝在不同狀態類別中，方便管理與擴充。

問題背景與需求

問題點（Problem）	說明
行為會隨狀態改變	Ticket system 在不同狀態下，行為不同（如投幣、退幣、購票等）。
大量條件判斷式	原本程式中，公開操作包含大量 if-else 判斷目前狀態，導致程式難讀且難維護。
允許外部直接切換狀態	客戶端希望能直接切換 Ticket system 的狀態，方便測試或回復狀態。

主要設計問題

- **如何設計 Ticket system 類別，使工程師能專注維護每個狀態的行為？**
- **如何讓 Ticket system 物件隨狀態改變，能抽換自身行為？**

狀態模式的解決方案

1. 封裝變動之處（Encapsulate what varies）

- **三個狀態類別**：
  - Instock（有票存）
  - Sold out（售罄）

Thetawave.ai

    - Enough Coins（硬幣足夠）
- 每個狀態類別封裝該狀態下特有的行為與狀態轉換操作（如按鈕事件）。
- 將狀態轉移事件定義為狀態類別的方法，例如 PressRefundButton、PressBuyButton。

2. 萃取共同行為（Abstract Common Behaviors）

- 建立一個抽象的 State 類別：
    - **抽象操作（Abstract Operations）**：InsertCoin、PressBuyButton（因不同狀態行為差異大）
    - **具體操作（Concrete Operations）**：EnterState、ExitState、PressRefundButton、FillTickets（行為較一致）
- 各子類別可覆寫抽象方法，實現各自的行為變異。
- 例如，Instock 狀態覆寫 InsertCoin 和 PressBuyButton，且標記 PressBuyButton 為不支援操作（Unsupported），若呼叫會印錯誤訊息。

3. 委派（Delegation）

- **Ticket system 作為 Context**，持有當前的 State 物件。
- 將所有會因狀態而變的公開操作交由當前 State 物件處理（委派）。
- State 物件可反向呼叫 Ticket system 的內部私有操作，如更新硬幣顯示、開關燈、吐幣、發票等。
- 私有操作權限設為 Package 權限，確保只有同一 Package 內的 State 類別可存取。

狀態生命周期管理

- 在 EnterState 方法中：
    1. 呼叫舊狀態的 ExitState 進行清理
    2. 更新當前狀態物件
    3. 呼叫新狀態的 EnterState 執行初始化動作
- 例如 Enough Coins 狀態：
    - 進入時會開燈（TurnLight(true)）
    - 離開時會關燈（TurnLight(false)）

各狀態類別實作重點

狀態類別	主要行為與規則
Sold out	- 投幣後立即吐回硬幣 - 按購買按鈕會出錯（無票可賣） - 放入票券後轉移到 Instock 狀態 - 進入時確保票數為 0，硬幣吐出
Instock	- 可接受投幣 - 按購買按鈕不合法（標記為 Unsupported） - 票數不能為 0 - 投幣數量不能超過票價，超過會吐幣
Enough Coins	- 投幣不合法 - 硬幣總額等於票價 - 按購買按鈕可購票 - 進入時開燈，離開時關燈 - 票數不可為 0，否則例外

Thetawave.ai

狀態模式的類別結構圖

類別	說明
TicketSystem (Context)	持有 State 物件，對外公開操作，行為委派給當前 State，並提供私有操作給 State 存取。
State (抽象類別)	定義抽象操作和具體操作，作為所有狀態類別的父類別。
InstockState	封裝有票狀態下的行為。
SoldOutState	封裝售罄狀態下的行為。
EnoughCoinsState	封裝硬幣足夠狀態下的行為。

狀態模式套用流程總結

步驟	說明
1. 列出狀態	明確定義所有可能狀態，並以屬性標示。
2. 繪製狀態機圖	捕捉公開操作的狀態轉移行為，方便分析狀態間轉換。
3. 依賴反轉三步驟 (重構)	(a) 封裝變動行為成不同狀態類別 (b) 萃取共同行為成抽象狀態類別 (c) Context 委派行為給 State 物件。

狀態模式的設計成果與優點

成果	說明
Client 可直接切換狀態	透過注入不同 State 物件，改變 TicketSystem 行為，方便測試與狀態還原。
行為封裝在狀態子類別中	各狀態的行為邏輯獨立，程式碼清晰且維護性高。
符合開放封閉原則（OCP）	新增狀態只需新增對應 State 類別，無需修改 TicketSystem 類別。
減少大量條件判斷式	TicketSystem 中不再撰寫複雜 if-else 判斷，委派給狀態物件處理。

難點釐清與重要解釋

- **狀態模式與狀態機的關係**
  狀態模式是以物件導向方式實作狀態機的設計模式。狀態機圖（State Machine Diagram）為分析工具，狀態模式為程式碼層級的具體實作。
- **Entry Action 與 Exit Action**
  狀態生命週期中的「進入狀態」與「離開狀態」動作，封裝於 EnterState 與 ExitState 方法中，實現狀態轉換時的必要行為，如開燈關燈、清理數據等。
- **雙向關聯設計**
  TicketSystem 與 State 之間為雙向關聯，TicketSystem 持有當前 State，State 持有 TicketSystem 的參考，方便狀態操作時呼叫 Context 的內部方法。
- **抽象類別操作的分類**
  將變化大的行為設為抽象方法，強制子類別覆寫；行為穩定的則做為具體方法提供預設實作。

Thetawave.ai

重要示意表格 — 狀態與操作對應

狀態類別	InsertCoin	PressBuyButton	PressRefundButton	FillTickets	EnterState 行為
InstockState	允許，增加硬幣	不合法（Unsupported）	退硬幣	增加票數，保持狀態	清理多餘硬幣，票數不能為0
SoldOutState	投幣後吐回硬幣	不合法，印錯誤訊息	退硬幣（0枚）	放票後轉換至 Instock	確保票數為0，吐出所有硬幣
EnoughCoinsState	不允許	允許，完成購票	退硬幣	不變	開燈

狀態模式設計的物件導向技術應用

技術	說明
封裝 (Encapsulation)	將狀態行為封裝於不同類別，減少程式碼耦合。
繼承 (Inheritance)	抽象 State 類別定義行為模板，具體狀態類別繼承並覆寫。
多型 (Polymorphism)	TicketSystem 呼叫狀態物件的同一方法，行為依狀態物件而異。
依賴反轉 (Dependency Inversion)	TicketSystem 不直接實作狀態行為，而依賴抽象 State，利於擴充與維護。

狀態模式將行為依狀態分散封裝，透過物件導向的多型機制委派行為，能有效降低複雜度與條件判斷的膨脹，提升系統維護性與擴充性。行為變動性由狀態類別管理，而 Context 類別則保持簡潔，兩者協同實現動態行為切換的需求。

5.1 景點 - 觀察者模式 _ 響應式行為變化.mp4

🕵️ 觀察者模式 (Observer Pattern) 概述與應用場景

> **觀察者模式** 是一種行為型設計模式，適用於當一個物件（稱為「主體」或 Subject）的狀態改變時，需要自動通知並更新多個相關物件（稱為「觀察者」或 Observer），使系統能即時反應狀態變化且保持資料一致性。

主要應用場景

- 當系統中存在多個需要對同一事件或狀態改變做出響應的行為（**多道響應式行為**，Reactive Behaviors）
- 希望系統未來能夠彈性擴充或刪除這些響應行為，而不需改動原有的主體類別，符合**開閉原則**（Open-Closed Principle, OCP）

🗂️ 案例背景：學員資料與圖表自動更新系統

需求說明

- 有一個檔案 "student.data"，包含 3000 筆學員資料。
- 必須建立兩種圖表：
  - **程式語言專長長條圖**（Student Language Bar Chart）
  - **職稱分布圓餅圖**（Student Job Title Pie Chart）

- 當學員資料檔案被修改時，圖表要自動更新並覆寫檔案。
- 未來可能新增更多不同類型的圖表。
- 需支援非工程師的資料維護者，讓系統自動維護資料與圖表一致性。

Thetawave.ai

初期實作問題點

- **Student Data File** 類別負責監控檔案變動，每秒讀取檔案並判斷是否有更動。
- 若有更動，則直接呼叫兩個圖表類別更新圖表，並新增備份檔案。
- 這導致 Student Data File 必須知道所有響應行為的具體類別，違反 **開閉原則**：
  - 每新增一種響應式行為，都必須修改 Student Data File 類別。
  - 程式耦合度高，維護困難。

🔧 觀察者模式解決方案詳解

1. 封裝變動點（Encapsulate What Varies）

- 將不同的響應式行為封裝在獨立的類別中：
  - Student Language Bar Chart（建立長條圖）
  - Student Job Title Pie Chart（建立圓餅圖）
  - Student Data File Backup（備份）
- 這些類別稱為**觀察者**，持續觀察 Student Data File 的狀態改變。

2. 抽象共同行為（Abstract Common Behaviors）

- 定義一個觀察者介面（Interface）StudentDataObserver，包含統一的 update() 方法。
- 所有具體觀察者類別實作此介面，並在 update() 方法中實作自己的響應行為。

> 引用：
> 「一個觀察者必須實作 update()，當被通知時，會從 Subject 取得最新狀態並做出回應。」

3. 委派通知責任（Delegation）

- Student Data File 類別維護一個觀察者清單（List），當狀態改變時，呼叫所有觀察者的 update()。
- Student Data File 只負責通知觀察者，**不直接執行響應行為**，實現解耦。

4. 依賴注入（Dependency Injection）

- 透過 register(observer) 和 unregister(observer) 方法動態增加或移除觀察者，提升彈性。
- 觀察者可在系統執行時依需求註冊或取消註冊，無需修改主體類別。

📊 Pull Model 與 Push Model 觀察者模式

模型名稱	通知方式說明	優點與缺點
Pull Model（拉模式）	主體只通知觀察者「狀態改變了」，觀察者自行向主體拉取最新資料。	- 觀察者與主體仍有依賴（因需取得資料） - 彈性較高，觀察者決定需哪些資料
Push Model（推模式）	主體在通知時直接將最新狀態或資料推送給觀察者。	- 觀察者與主體解耦較徹底 - 需設計好推送資料內容，否則資料不足或過多

Thetawave.ai

案例中使用的 Pull Model

- 觀察者的 update() 方法會從 Student Data File 索取最新學員資料，再繪製圖表或進行備份。
- Concrete Subject（Student Data File）不需知道觀察者的具體行為。

推模式的優劣

- 推模式可降低觀察者與主體的耦合度。
- 但推送的資料需謹慎設計，避免不足或過多，否則觀察者仍需向主體索取部分資料。

觀察者模式中主要角色與結構

角色名稱	職責說明
Subject（主體）	維護狀態，並管理觀察者清單，狀態變動時通知觀察者
Observer（觀察者）	實作 update() 方法，回應主體狀態改變，執行特定響應行為
Concrete Subject	主體的具體實作，管理具體狀態與通知細節
Concrete Observer	具體觀察者，實作特定的響應行為（繪製圖表、備份資料等）

觀察者模式實作流程總結

1. **定義觀察者介面**：包含統一的 update() 方法。
2. **主體類別維護觀察者清單**：提供 register()、unregister() 方法管理觀察者。
3. **主體狀態改變時通知觀察者**：呼叫所有註冊觀察者的 update()。
4. **觀察者根據需要取得主體最新狀態並做出反應**：例如繪製圖表、備份檔案。
5. **擴充響應行為只需新增觀察者類別並註冊**：無需修改主體類別，符合開閉原則。

重要觀念與易混淆點說明

觀念	解釋與說明
多道響應式行為	指系統中對同一狀態改變，有多種不同的行為需要被觸發，如繪製不同圖表、備份資料等。
開閉原則（OCP）	軟體實體應對擴充開放，對修改關閉。觀察者模式讓主體類別無需修改即可擴充新的響應行為。
耦合度	觀察者模式降低主體與觀察者之間的耦合度，使系統更易維護與擴展。
Pull Model 與 Push Model	Pull Model 中觀察者主動拉取資料，Push Model 中主體主動推送資料。各有優缺點須依需求選擇。
依賴注入（Dependency Injection）	透過方法注入觀察者，使主體可動態管理觀察者，提升系統彈性。

觀察者模式與軟體設計原則的關聯

原則	觀察者模式如何體現

Thetawave.ai

單一職責原則（SRP）	將不同響應行為封裝在獨立觀察者類別，避免主體類別負責過多事情。
開閉原則（OCP）	新增響應行為只需新增觀察者並註冊，主體類別不需修改。
依賴反轉原則（DIP）	主體依賴抽象觀察者介面而非具體實作，降低耦合。
封裝變動（Encapsulate What Varies）	將會變動的響應行為封裝起來，使系統彈性增加。

🛠️ 具體實作建議與設計細節

觀察者介面範例（偽碼）

```
interface StudentDataObserver {
    void update();
}
```

主體類別學生資料檔案（Student Data File）核心方法

方法名稱	功能說明
register()	註冊一個觀察者
unregister()	取消註冊觀察者
notifyObservers()	通知所有註冊的觀察者執行 update()
startMonitoring()	啟動檔案監控緒，偵測資料變動並通知

觀察者類別實作重點

- 在 update() 方法中：
  - 從 Student Data File 取得最新學員資料
  - 執行專屬的處理，如重繪圖表或備份檔案

📝 觀察者模式的決策判斷依據

判斷條件	是否適用觀察者模式	理由說明
系統中有多個響應式行為需要對同一事件做出反應	是	需要多對一通知機制
響應行為數量會隨系統擴充而改變	是	可透過註冊新觀察者擴充響應行為，符合開閉原則
響應行為固定且不會改變	否	無需額外複雜度，直接硬編碼即可
希望降低主體與響應行為的耦合度	是	觀察者模式強調解耦設計

🎯 觀察者模式對系統設計的影響

- **解耦合**：主體與具體響應行為分離，主體只負責狀態管理與通知。
- **彈性擴充**：新增或移除響應行為只需新增/註銷觀察者，無需變更主體類別。
- **符合開閉原則**：系統對修改關閉、對擴充開放，降低維護風險。
- **易於維護與測試**：各觀察者獨立測試，故障不互相影響。
- **提升系統反應速度**：狀態變更即時通知，保證資料與狀態同步。

觀察者模式透過分離主體與多重響應行為，實現系統的高內聚、低耦合，並使系統能有效因應未來的功能擴展與變更，這在實務設計中極其重要。

Thetawave.ai